

FinPilot: Architecture and Design Report

Investment portfolio and goal monitoring for an internal wealth-service team. All data is synthetic.

Repository state	branch <code>feat/admin-import-reports</code> at <code>f303d81</code> (API complete), plus this report
Status of parts	API, database, import pipeline, tests and API CI: implemented and described from the code. Web UI (Next.js), container images, the <code>docker.yml</code> workflow and the TLS profile were being built in parallel when this was written; they are described as the agreed design and marked (<i>design</i>)
Companion documents	<code>DECISIONS.md</code> (assumptions, omissions, ADRs), <code>docs/performance/query-plans.md</code> (EXPLAIN evidence), <code>docs/sql/reviewer-queries.sql</code> (SQL tasks), <code>docs/api/openapi.json</code> (OpenAPI 3.1)

1. Executive overview

Problem. A service user needs one consolidated, trustworthy view of a customer's accounts, positions, valuation, transactions, risk profile and goals; an operations administrator needs to load daily CSV files without creating duplicates or silently accepting bad rows. The supplied data is deliberately imperfect, so the system has to say what it rejected and why.

Scope delivered. Cookie-session login with VIEWER/ADMIN roles; customer search and profile with latest risk assessment; server-side portfolio valuation (account, asset-class and customer totals, positions, unrealised P&L, data-freshness dates); filtered, sorted, paginated transaction ledger; goal list/create/edit with validation and consistency flags; admin CSV import of transactions with staging, SQL-side validation, idempotent merge and a row-level error report; a firm-wide overview answering the reviewer SQL tasks; health endpoint, structured logs with request correlation, CI.

Out of scope by design. Order execution, payments, investment advice, real-time prices, notifications, multi-tenancy, Kubernetes, microservices (see `DECISIONS.md`).

Stack.

Layer	Choice	Why, in one line
Database	PostgreSQL 16	Constraints, <code>NUMERIC</code> , <code>GROUPING SETS</code> , <code>pg_trgm</code> , <code>COPY</code> , advisory locks: the domain rules live where the data lives
API	Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2 async + asyncpg, Alembic	Typed contracts become the OpenAPI spec; async pool; <code>COPY</code> via <code>asyncpg</code> ; migrations with autogenerate drift check
Auth	Argon2id (argon2-cffi), PyJWT HS256 in an HttpOnly cookie	No token reachable by page scripts; stateless verification plus a per-request user re-read
Web (<i>design</i>)	Next.js 15 App Router, TypeScript strict, Tailwind + shadcn/ui, TanStack Query, React Hook Form + Zod, Recharts	Same-origin proxy keeps the cookie model simple; server state is cached, not re-implemented
Delivery	Docker Compose (db, api, web), GitHub Actions	One-command local stack; CI gates on lint, types, migrations and tests against real PostgreSQL

Major design decisions (full ADRs in `DECISIONS.md`):

- The database enforces the domain.** Natural business keys as primary keys, `VARCHAR + CHECK` vocabularies, `NUMERIC` money, composite and conditional `CHECKs` (e.g. BUY/SELL must have quantity and price > 0; DIVIDEND/FEE must have both = 0). Application validation mirrors these for good error messages; the constraints are the last line of defence.
- All money arithmetic in PostgreSQL `NUMERIC`.** Valuation is a view; aggregation is SQL (`GROUPING SETS`, window functions). Python only shapes results and rounds half-up to 2 dp at serialisation.
- Import = stage, validate in SQL, merge, in one transaction; partial acceptance.** Good rows load, bad rows are quarantined in `import_errors` with line number, code, message and the raw row. Re-import is idempotent; a changed row with a known id is rejected, never overwritten.
- Portfolio is valued from the holdings snapshot, not derived from the ledger,** because the supplied ledger does not reconcile to the snapshot (section 4.6).
- Same-origin deployment.** The browser only talks to the web origin; Next.js rewrites `/api/v1/*` to the API, so the session cookie is first-party and CORS is not on the hot path.

2. System context

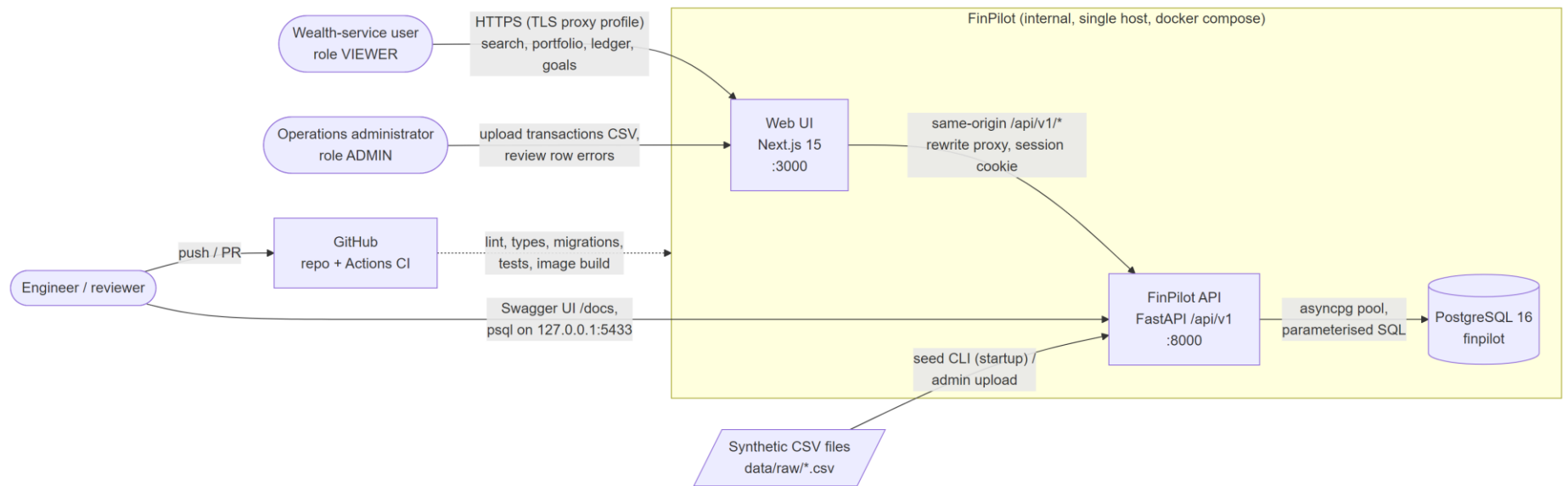


Figure: System context

Actor / system	Interaction
Wealth-service user (VIEWER)	Searches customers, reads portfolio/ledger/overview, creates and edits goals
Operations administrator (ADMIN)	Everything a VIEWER can do, plus transaction CSV upload and import history
Engineer / reviewer	Swagger UI at :8000/docs, psql on 127.0.0.1:5433, GitHub Actions results
Synthetic CSVs (data/raw/)	Loaded by the idempotent seed command at API start-up; transactions also via upload
External dependencies	None at runtime: no market-data feed, e-mail, identity provider or cloud service. Build-time: PyPI/npm registries, Docker Hub, GitHub

3. Component architecture

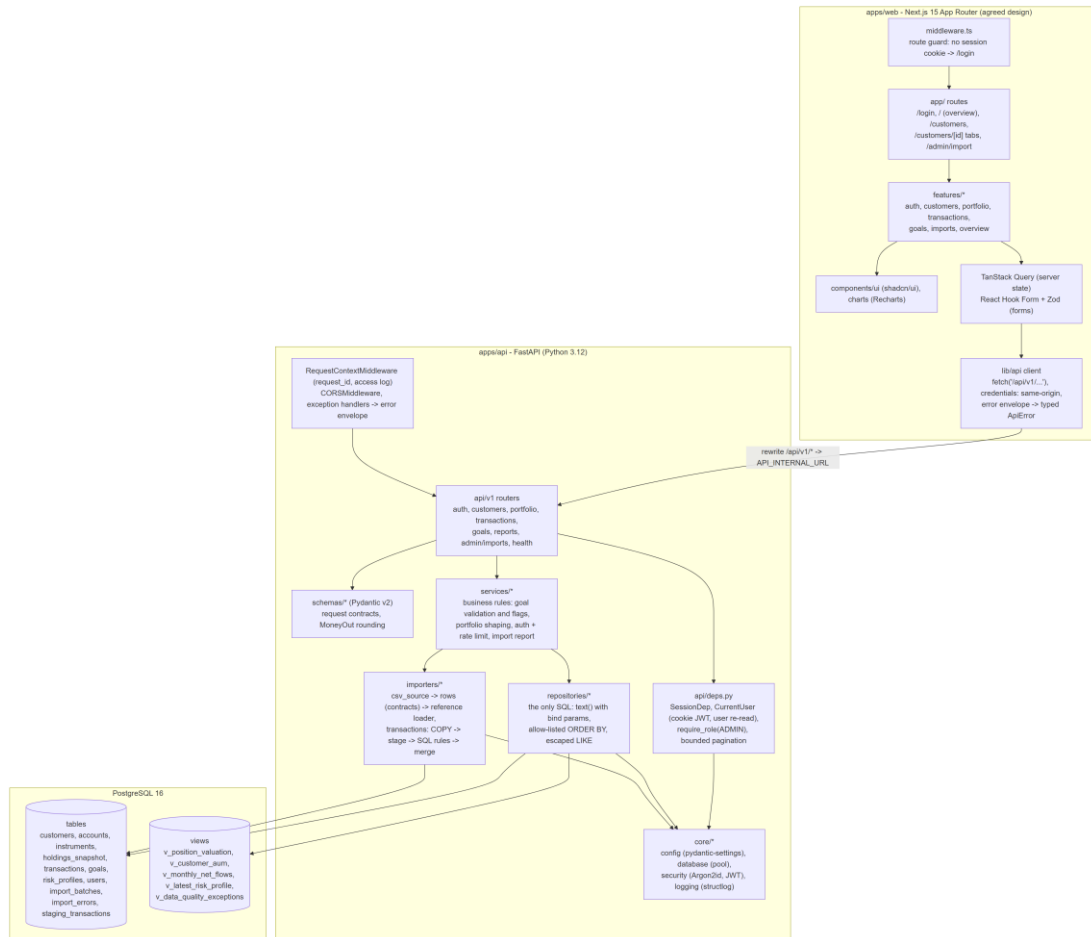


Figure: Component architecture

3.1 Backend layers (implemented)

Layer	Package	Responsibility	Must not
Transport	app/api/v1/*.py , app/api/deps.py	HTTP mapping, status codes, request/response models, auth dependencies (CurrentUser , AdminUser), bounded pagination params, path regexes	Contain SQL or business rules
Contracts	app/schemas/*	Pydantic models: extra="forbid" on writes, enums, decimal precision limits, MoneyOut serialisation	Touch the database
Services	app/services/*	Business rules: goal validation on the merged state and flags, portfolio shaping (every account listed, freshness), login flow + rate limit, import report assembly	Build SQL strings
Data access	app/repositories/*	The only place SQL lives. Parameterised text() or SQLAlchemy Core; allow-listed ORDER BY ; one round trip per concern	Make HTTP decisions
Ingestion	app/importers/*	CSV parsing with physical line numbers, row contracts, reference loader, transaction stage/validate/merge, batch lifecycle	Depend on FastAPI
Core	app/core/*	Settings, pooled engine, error envelope, request-id middleware, structured logging, password hashing and JWT	Know about the domain

The dependency direction is strictly inward: routers -> services -> repositories -> core. The importers are reused unchanged by the HTTP endpoint and by the CLI (`python -m app.cli seed`), which is why the seed and an admin upload produce identical audit records.

Data-access boundary. Repositories return `RowMappings` or ORM rows; services convert them to Pydantic models.

Aggregates are computed by views (`v_position_valuation`, `v_customer_aum`, `v_monthly_net_flows`) so the same definition backs the API, the overview report and the reviewer SQL. There are no per-row queries: the portfolio endpoint issues 5 queries regardless of position count, the customer list 1 (page + total via `count(*) OVER ()`).

3.2 Frontend modules *(design)*

Module	Content
<code>middleware.ts</code>	Route guard: no session cookie -> redirect to <code>/login</code> (authorisation is still enforced by the API)
<code>app/ routes</code>	<code>/login</code> , <code>/ overview</code> , <code>/customers</code> , <code>/customers/[id]</code> with Overview / Portfolio / Transactions / Goals tabs, <code>/admin/import</code>
<code>features/*</code>	<code>auth</code> , <code>customers</code> , <code>portfolio</code> , <code>transactions</code> , <code>goals</code> , <code>imports</code> , <code>overview</code> : each owns its query hooks, components and Zod schemas
<code>lib/api</code>	Typed fetch wrapper calling same-origin <code>/api/v1/*</code> ; maps the error envelope to a typed error (field errors -> form fields, 401 -> login)
State	TanStack Query for all server state (cache keys per customer/filter set; <code>keepPreviousData</code> for pagination); URL search params for ledger filters; no global client store
UI	shadcn/ui primitives, Recharts allocation chart with a table fallback, status badges for PENDING/REVERSED

4. Data model

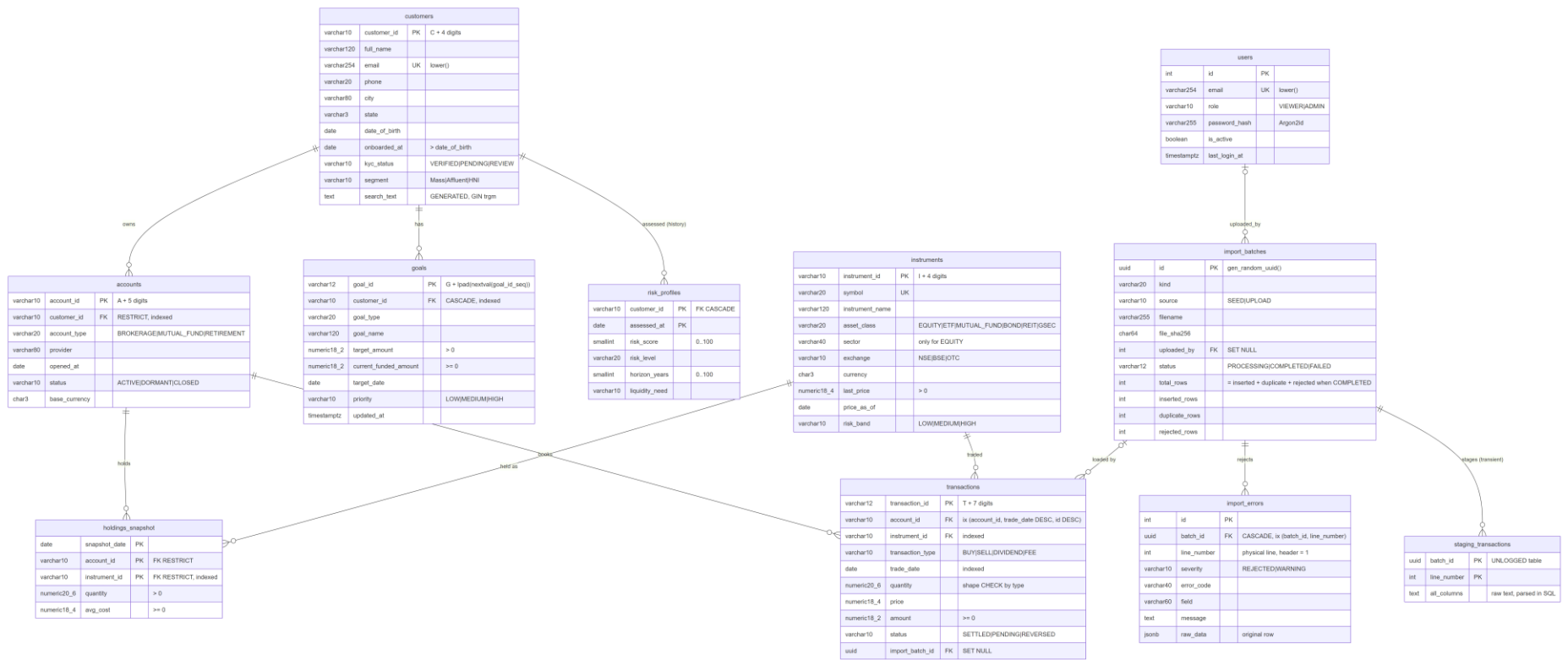


Figure: Entity-relationship diagram

4.1 Tables, keys and cardinalities

Table	Primary key	Relationships	Notes
customers	customer_id (C0001)	1 -> 0..n accounts, goals, risk_profiles	email unique and lower-case; generated search_text
accounts	account_id	n -> 1 customer (RESTRICT)	status ACTIVE/DORMANT/CLOSED
instruments	instrument_id	1 -> 0..n holdings, transactions	symbol unique; sector required iff EQUITY
holdings_snapshot	(snapshot_date, account_id, instrument_id)	n -> 1 account, instrument (RESTRICT)	composite key makes a duplicate position impossible
transactions	transaction_id	n -> 1 account, instrument (RESTRICT); n -> 0..1 import batch (SET NULL)	ledger; import_batch_id = lineage
goals	goal_id = 'G'		lpad(nextval('goal_id_seq'),5,'0')
risk_profiles	(customer_id, assessed_at)	n -> 1 customer (CASCADE)	history kept; v_latest_risk_profile picks the latest
users	id (identity)	1 -> 0..n import batches (SET NULL)	role VIEWER/ADMIN, Argon2id hash
import_batches	id (uuid)	1 -> 0..n import_errors (CASCADE)	one row per file load, seed or upload
import_errors	id	n -> 1 batch	line_number (physical, header = 1), error_code, field, message, raw_data JSONB
staging_transactions	(batch_id, line_number)	transient	UNLOGGED, every column TEXT

ON DELETE is RESTRICT for financial facts (an account with history cannot vanish) and CASCADE only for customer-owned descriptive data (goals, risk profiles).

4.2 Constraints worth noting

- Types:** money NUMERIC(18,2), prices NUMERIC(18,4), quantities NUMERIC(20,6); dates are DATE, audit times TIMESTAMPTZ.
- Vocabularies:** VARCHAR + CHECK (x IN (...)) for every enumeration (kyc_status, segment, asset_class, exchange, transaction_type, status, priority, role, ...).
- Identifier formats:** CHECK (customer_id ~ '^C[0-9]{4,}\$') and equivalents.
- Cross-column:** onboarded_at > date_of_birth; (asset_class = 'EQUITY') = (sector IS NOT NULL); transaction shape by type; amount >= 0; quantity > 0 in holdings; goal target_amount > 0, current_funded_amount >= 0.
- Audit integrity:** import_batches CHECK (total_rows = inserted_rows + duplicate_rows + rejected_rows OR status <> 'COMPLETED'): a completed batch whose counters do not reconcile cannot be written.

4.3 Indexes and the query each one serves

Indexes are created for measured query patterns only (evidence in query-plans.md).

Index	Serves	Evidence (3M-row dataset)
ix_transactions_account_date (account_id, trade_date DESC, transaction_id DESC)	Ledger page for a customer's accounts in the default order; count(*) as index-only scan; single-account filter returns rows pre-ordered, no sort	page 362 -> 1.3 ms; count 280 -> 0.2 ms
ix_customers_search_trgm GIN (search_text gin_trgm_ops)	search_text LIKE '%term%' (id, name, e-mail, city)	29 -> 0.21 ms
ix_accounts_customer_id	Every customer-scoped read (portfolio, ledger scope, profile counts)	portfolio 18 -> 0.39 ms
pk_holdings_snapshot	max(snapshot_date) (index-only, backward) and positions of an account on a date	used by every portfolio read
ix_transactions_trade_date	Date-bounded reporting (monthly flows)	1,276 -> 414 ms with bound pushed down
ix_transactions_instrument_id, ix_holdings_snapshot_instrument_id	Ledger instrument filter, holder counts, FK checks when an instrument is deleted	-
ix_goals_customer_id, ix_import_batches_kind (kind, started_at DESC),	Goal list; import history; error pages ordered by line	-

ix_import_errors_batch_id_line_number		
---------------------------------------	--	--

4.4 Views (plain, not materialized)

View	Definition / purpose
v_position_valuation	holdings x accounts x instruments with cost_basis , market_value , unrealized_pnl in NUMERIC . Inlined by the planner, so customer predicates reach the indexes
v_customer_aum	Per-customer totals at max(snapshot_date)
v_latest_risk_profile	DISTINCT ON (customer_id) ... ORDER BY assessed_at DESC
v_monthly_net_flows	SETTLED BUY/SELL per month, FILTER aggregates, net = BUY - SELL
v_data_quality_exceptions	UNION ALL of import rejections and reconciliation rules (trade before account opened, position in inactive account, goal over-funded / overdue / name-type mismatch)

They are plain views because at this volume they are always fresh and cost milliseconds; the measured scale limits (customer list AUM, monthly flows) and their fixes are in section 10.

4.5 Money and rounding

Values are exact **NUMERIC** in the database and **Decimal** in Python; each figure is rounded half-up to 2 dp only when serialised. Because every figure is rounded independently, a sum of rounded parts can differ from the rounded total by up to 0.01 per part; the OpenAPI description states this. Amounts are emitted as JSON numbers for chart consumers (a trade-off noted in **DECISIONS.md**).

4.6 Data-quality findings and handling

Discovered by profiling the raw files and pinned by tests (**test_import_pipeline.py**). Line numbers are physical file lines, header = line 1.

File	Finding	Rows	Handling	Rationale
holdings_sn apshot.csv	Exact duplicate positions: A00147/I0049 (line 984), A00145/I0036 (985), A00146/I0010 (986)	3	Rejected DUPLICATE_IN_FILE ; first occurrence (lines 836, 830, 833) kept	Summing would double-count market value; the composite PK makes the rule structural. 985 - > 982 loaded
transaction s.csv	T0000026 repeated (line 4552; first on line 27)	1	Rejected DUPLICATE_IN_FILE , first occurrence kept	An id identifies one economic event
transaction s.csv	T0004551 references instrument I9999 (4553)	1	Rejected UNKNOWN_INSTRUMENT	Cannot value or display an unknown security
transaction s.csv	T0004552 FEE with amount - 75.00 (4554)	1	Rejected NEGATIVE_AMOUNT	Direction is carried by transaction_type ; a signed amount would double-negate. Not auto-corrected: the sign may be a data-entry error in either field
transaction s.csv	T0004553 trade date 2027-01- 05 (4555)	1	Rejected FUTURE_TRADE_DATE (after the import business date)	A future trade is not a fact yet
transaction s.csv	Trades dated before their account's opened_at	1,069	Loaded, flagged (TRADE_BEFORE_ACCOUNT_OP ENED in v_data_quality_exceptio ns)	Plausible migration/back-dating artefact; rejecting 23 % of the ledger would hide more than it protects. Needs a business owner decision
transaction s.csv	Trades on CLOSED/DORMANT accounts	311 loaded (313 in file; 2 are among the rejected rows)	Loaded , visible via reviewer query Q6d	A closed account can have historic trades. Not yet a rule in the exceptions view
goals.csv	goal_name describes a different goal_type (e.g. type TRAVEL, name "Personal Education")	149 of 177	Loaded, flagged NAME_TYPE_MISMATCH (API flag + view)	The type drives behaviour; the name is free text. Flag for review, do not guess which is right
holdings vs transaction s	Snapshot quantities do not reconcile to the ledger: 0 of 982 positions equal settled net BUY - SELL quantity (only 225	all	Design decision: portfolio valued from the snapshot; ledger presented as history; reconciliation query Q6e	The ledger is evidently a sample, not the full history. Deriving positions from it would produce negative holdings

	positions have any trades); 1,187 of 3,068 account/instrument pairs would be net short		documents the gap	
--	---	--	-------------------	--

Result on the supplied data: transactions 4,554 -> 4,550 loaded, 4 rejected; holdings 985 -> 982, 3 rejected; all other files load fully. Total AUM 52,754,963.60 INR across 113 customers with positions (7 customers have accounts but no positions and get a zero portfolio, not an error); top customer C0026 (1,953,574.59).

5. API design

5.1 Endpoint map

All under `/api/v1`. Every endpoint except health and login requires the session cookie.

Method	Path	Role	Purpose	Notes
GET	<code>/health</code>	public	Liveness + DB readiness	200 / 503, exposes no secrets
POST	<code>/auth/login</code>	public	Verify credentials, set cookie	200, 401, 422, 429
POST	<code>/auth/logout</code>	public	Clear cookie	204
GET	<code>/auth/me</code>	any	Signed-in user	401
GET	<code>/customers</code>	any	Search (<code>search</code> , <code>kyc_status</code> , <code>segment</code>), <code>sort</code> , <code>page</code> , <code>page_size</code>	includes account count and AUM
GET	<code>/customers/{customer_id}</code>	any	Profile + latest risk	404 <code>CUSTOMER_NOT_FOUND</code>
GET	<code>/customers/{customer_id}/portfolio</code>	any	Totals, accounts, allocation, positions, freshness	example in OpenAPI
GET	<code>/customers/{customer_id}/transactions</code>	any	<code>date_from</code> , <code>date_to</code> , <code>account_id</code> , <code>instrument_id</code> , repeated <code>transaction_type/status</code> , <code>sort</code>	422 for inverted dates or foreign account
GET	<code>/customers/{customer_id}/transactions/facets</code>	any	Filter options with counts, date range	
GET	<code>/customers/{customer_id}/goals</code>	any	Goals + funded % + flags + summary	
POST	<code>/customers/{customer_id}/goals</code>	any	Create	201 + <code>Location</code> , id from DB sequence
PATCH	<code>/goals/{goal_id}</code>	any	Partial update	rules on merged state
GET	<code>/reports/overview</code>	any	Firm headline, top customers, allocation, flows, holders, underfunded goals, DQ counts	
POST	<code>/admin/imports/transactions</code>	ADMIN	Multipart CSV import	201 report, 403, 413, 415, 422; example in OpenAPI
GET	<code>/admin/imports</code> , <code>/admin/imports/{id}</code> , <code>/admin/imports/{id}/errors</code>	ADMIN	History, one batch, paged row errors	

5.2 Conventions

- **Authentication:** HttpOnly cookie `finpilot_session` carrying a JWT (section 8). Swagger UI works by calling login first; the browser then sends the cookie.
- **Validation:** Pydantic at the edge (path regexes such as `^C\d{4,}$`, enums, `max_length`, decimal precision, `extra="forbid"` on writes, explicit `null` rejected on PATCH); domain rules in services (funded $\leq$ target on the merged row; a *changed* `target_date` must be in the future, so an overdue goal can still record funding); database `CHECKs` as backstop.
- **Pagination:** `page` (1-based, $\leq 100,000$) and `page_size` (≤ 100) are bounded so no request can materialise a table; responses carry `{page, page_size, total, pages}`. Ordering always ends in a unique key, so pages never overlap or skip (tested).

- **Errors:** one envelope for every non-2xx, including framework 404/405 and validation errors. Unhandled exceptions return a generic 500; the stack trace goes only to the log, under the same `request_id` that is echoed in `X-Request-ID`.

HTTP/1.1 422 Unprocessable Content
X-Request-ID: req_3f2a9c1b7d4e5f60

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "current_funded_amount cannot exceed target_amount",
    "request_id": "req_3f2a9c1b7d4e5f60",
    "details": [
      {
        "field": "current_funded_amount",
        "message": "current_funded_amount cannot exceed target_amount"
      }
    ]
  }
}
```

Codes are stable and specific (`CUSTOMER_NOT_FOUND`, `GOAL_NOT_FOUND`, `INVALID_CREDENTIALS`, `SESSION_EXPIRED`, `RATE_LIMITED`, `FORBIDDEN`, `PAYLOAD_TOO_LARGE`, `FILE_HEADER_INVALID`, ...) so the UI can branch on `code` rather than parse messages.

- **Versioning:** URI prefix `/api/v1`. Additive changes (new fields, endpoints) stay in v1; a breaking change gets `/api/v2` served side by side until clients move. The OpenAPI 3.1 document is generated from the code and committed (`docs/api/openapi.json`), so contract changes show up in review diffs.

6. Workflows

6.1 Login and session

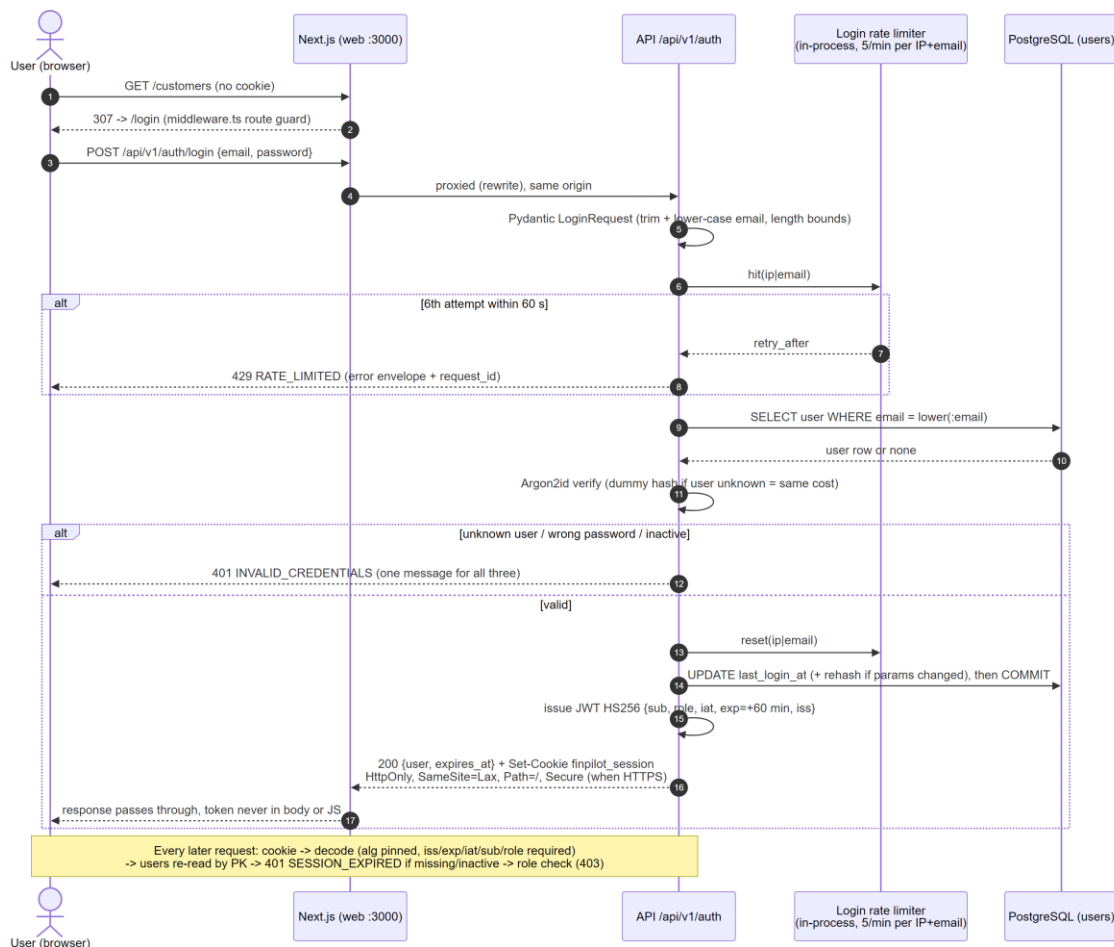


Figure: Login sequence

6.2 Customer portfolio read

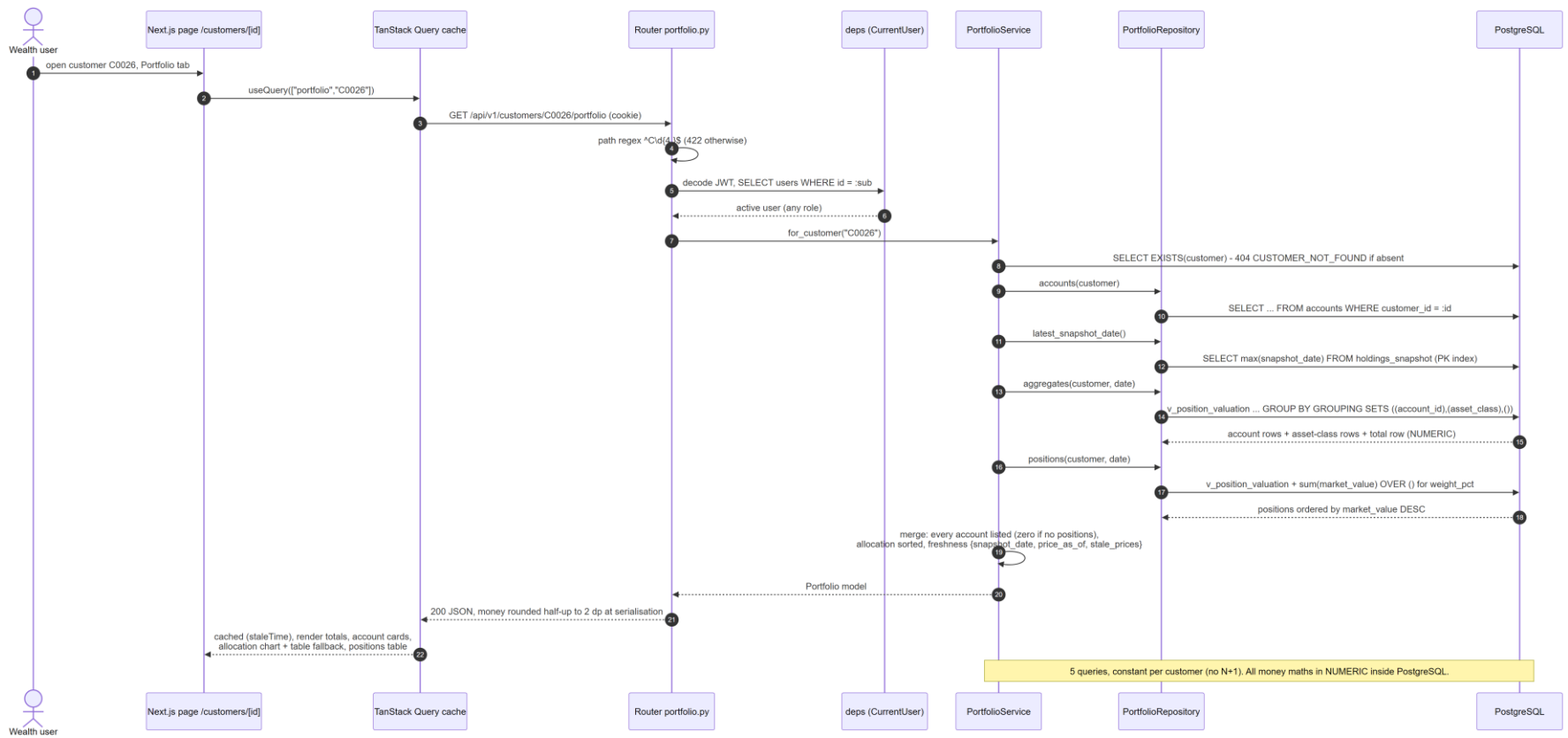


Figure: Portfolio read sequence

6.3 Goal update

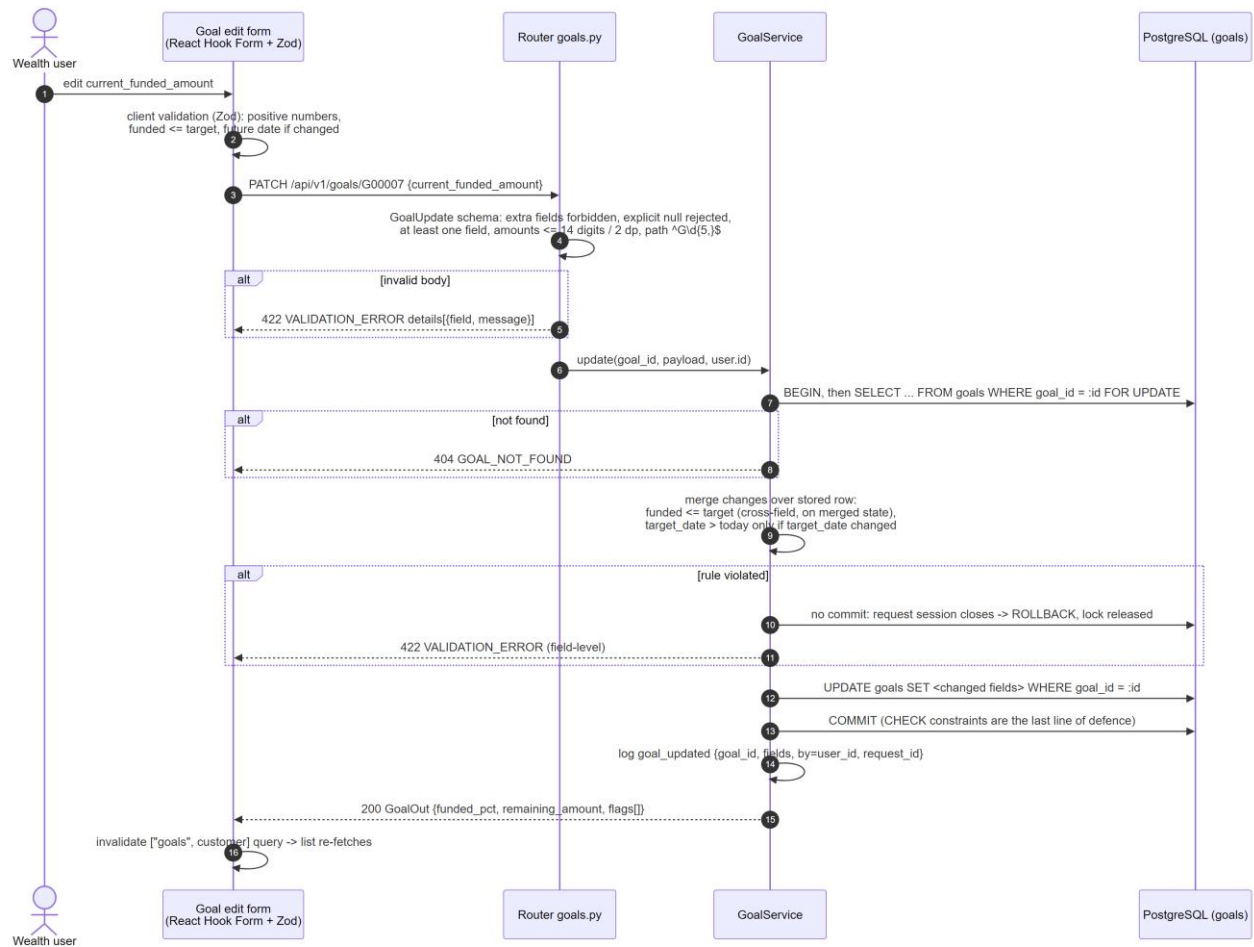


Figure: Goal update sequence

SELECT ... FOR UPDATE serialises two concurrent edits of the same goal, so cross-field rules are always checked against the committed row. It does not detect a *stale* edit (user B saving over user A's change made after B loaded the form); optimistic concurrency is on the roadmap.

6.4 CSV import (transactions)

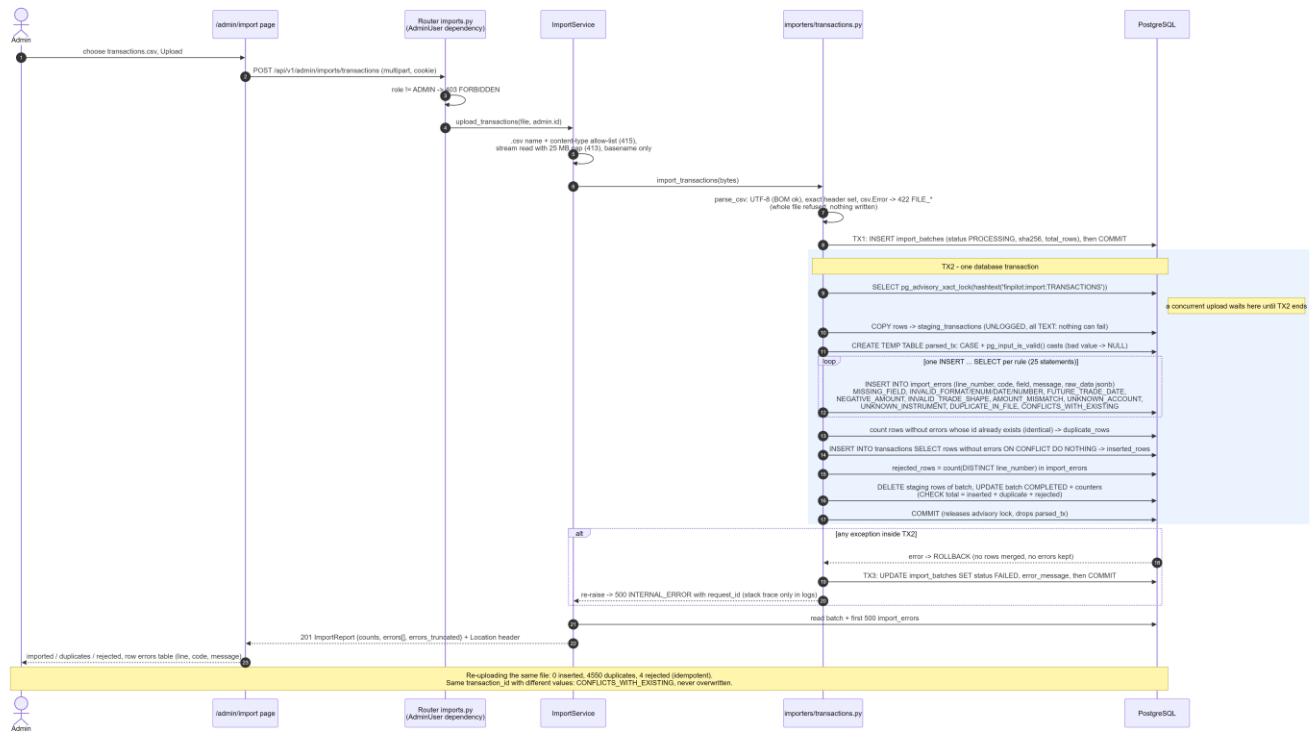


Figure: CSV import sequence

Properties this design guarantees, each covered by a test:

- **Structural failure = whole file refused** (bad encoding, wrong header, unparseable CSV, > 25 MB, non-CSV): 413/415/422, nothing written.
- **Row failure = partial acceptance:** every violated rule is recorded (a row can carry several reasons); the remaining rows load.
- **Idempotent:** an identical existing row is counted as a duplicate; re-uploading the supplied file yields **inserted 0, duplicates 4,550, rejected 4**.
- **No silent overwrite:** same **transaction_id**, different values -> **CONFLICTS_WITH_EXISTING**.
- **Concurrency:** **pg_advisory_xact_lock** serialises transaction imports so duplicate/conflict classification is exact; the primary key is the backstop.
- **Auditability:** the batch row is committed before the data transaction; if the merge fails it rolls back completely and the batch is marked **FAILED** with the error message.
- **Why SQL-side validation:** FK and duplicate checks are set operations against tables that grow to millions of rows; one **INSERT ... SELECT** per rule is a join, not a Python loop over ids. Casting is guarded with **pg_input_is_valid()**, so a malformed value becomes a reported error, never an aborted transaction.

Reference files (customers ... holdings) use a sibling path: Pydantic row contracts, in-file duplicate detection (first wins), batched FK checks, **INSERT ... ON CONFLICT DO NOTHING**, one transaction per file. They are loaded by the seed command only; uploading them is not exposed.

6.5 CI/CD and deployment

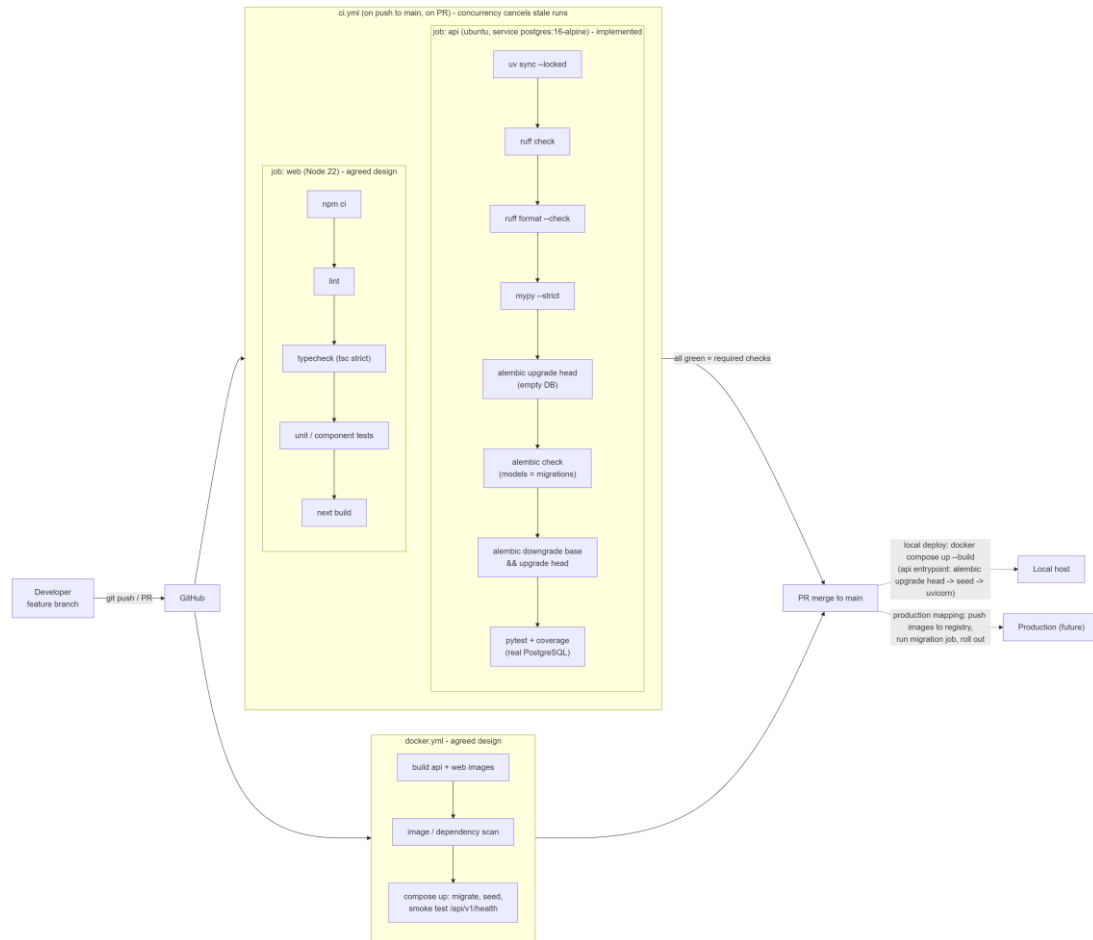


Figure: CI/CD pipeline

The `api` job is implemented in `.github/workflows/ci.yml` and runs on every PR and push to `main`: `uv sync --locked -> ruff check -> ruff format --check -> mypy --strict -> alembic upgrade head` on an empty PostgreSQL 16 service `-> alembic check` (models and migrations in sync) `-> alembic downgrade base && alembic upgrade head` (rollback path proven) `-> pytest` with coverage against that real database. Any failing step fails the pipeline. The `web` job and `docker.yml` (image build, scan, compose smoke test) are *(design)*.

7. Deployment topology

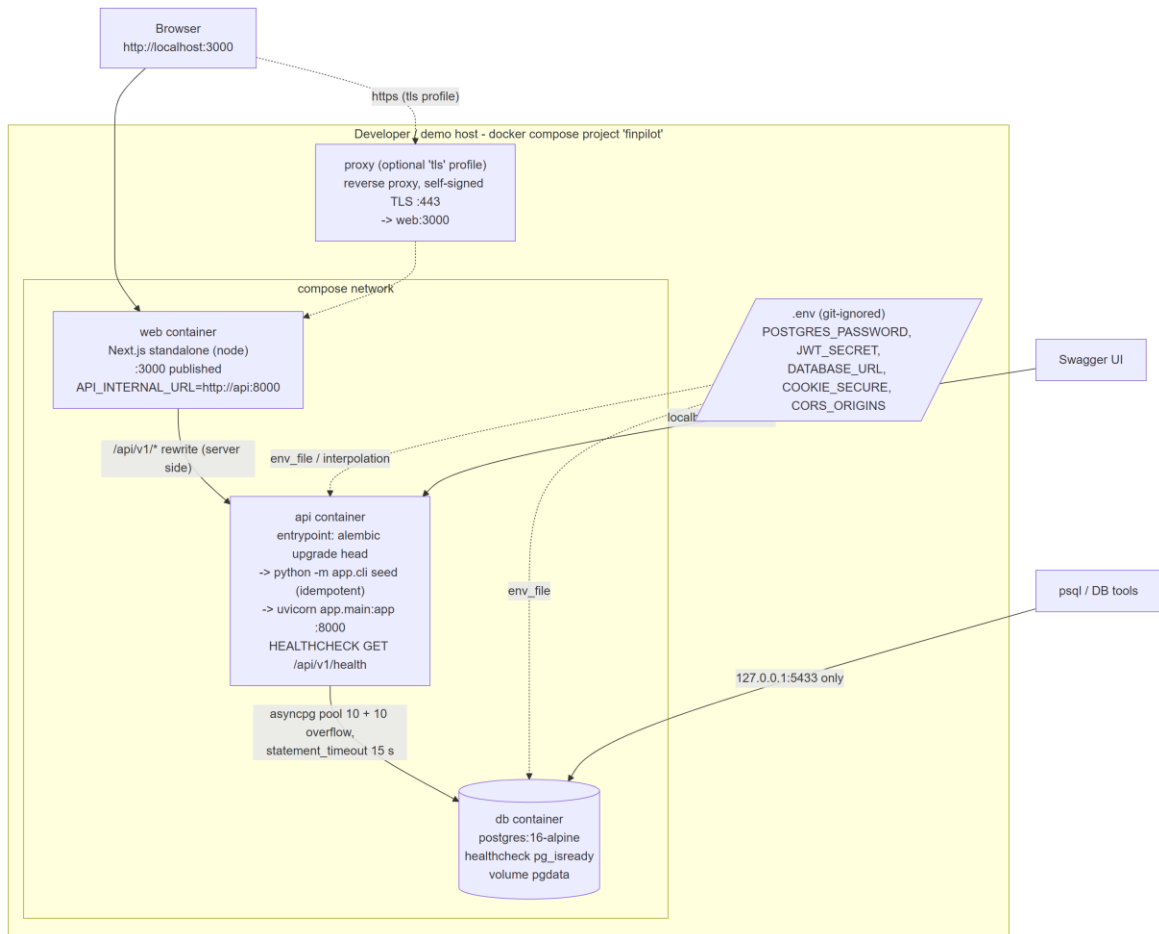


Figure: Deployment topology

Service	Image / runtime	Port (host)	Start-up	Health
db	postgres:16-alpine, volume pgdata	127.0.0.1:5433 only	-	pg_isready (implemented)
api (design)	Python 3.12 slim, non-root, uv locked deps	8000	alembic upgrade head -> python -m app.cli seed (idempotent) -> uvicorn	HEALTHCHECK GET /api/v1/health; starts after db is healthy
web (design)	Next.js standalone output on Node 22	3000	node server.js, API_INTERNAL_URL=http://api:8000	depends on api healthy
proxy (design, optional 'tls' profile)	reverse proxy with self-signed certificate	443	-	-

Secrets and configuration. Everything comes from environment variables (pydantic-settings); `.env` is git-ignored and `.env.example` documents every key with placeholder values. `DATABASE_URL` is read by both the API and Alembic (alembic.ini contains no URL). `JWT_SECRET` must be at least 32 characters or the API refuses to start. Demo users are created by the seed only when `ENVIRONMENT != production`.

Environments. `local` (console logs), `test` (pytest creates and migrates `finpilot_test`), `ci` (GitHub service container), `production` (JSON logs, no demo users, `COOKIE_SECURE=true`).

Production mapping.

Local	Production
compose db container	Managed PostgreSQL 16 (e.g. RDS / Cloud SQL / Azure Flexible Server): Multi-AZ, automated backups + PITR, private subnet, TLS required
api container	Same image on a container service (ECS/Cloud Run/AKS) behind an internal load balancer; >= 2 replicas;

	migrations run as a one-off release job before rollout, not by every replica
web container	Same image, or the Next.js app on a platform with the <code>/api/v1</code> rewrite pointing at the internal API
optional TLS proxy	Managed load balancer / ingress with a real certificate, HSTS, WAF rules; corporate SSO in front for an internal tool
<code>.env</code>	Secrets manager (AWS Secrets Manager / Vault / Key Vault) injected at runtime; rotated; separate DB roles for migration (DDL) and runtime (DML only)
stdout logs	Log shipping to a central store (CloudWatch/ELK/Loki) with retention and PII controls

8. Security

Concern	Control in this implementation
Password storage	Argon2id (argon2-cffi defaults, RFC 9106 low-memory profile); hashes upgraded transparently on login when parameters change
User enumeration	Identical 401 <code>INVALID_CREDENTIALS</code> for unknown user, wrong password and inactive user; identical responses are tested; a dummy hash is verified for unknown e-mails so response time does not reveal existence either
Brute force	5 attempts / 60 s per client IP + e-mail, sliding window, 429 with retry time; counter resets on success. In-process (see limitations)
Session token	JWT HS256, <code>alg</code> pinned on decode (no <code>none</code> /alg confusion), <code>iss</code> , <code>exp</code> , <code>iat</code> , <code>sub</code> , <code>role</code> required; 60-minute expiry; delivered only as <code>HttpOnly</code> ; <code>SameSite=Lax</code> ; <code>Path=/</code> cookie (<code>Secure</code> when served over HTTPS), never in a response body or <code>localStorage</code> , so XSS cannot exfiltrate it
Revocation	The user row is re-read on every request: deactivation or role change takes effect immediately, at the cost of one PK lookup
Authorisation	Role checks are FastAPI dependencies (<code>AdminUser = require_role(ADMIN)</code>) declared on the route, not ad-hoc <code>ifs</code> ; 401 vs 403 distinguished. VIEWER can read and maintain goals; only ADMIN can import (tested both ways)
Input validation	Pydantic contracts on every input; path/query regexes; enums; bounded lengths and page sizes; decimal precision; <code>extra="forbid"</code> on writes
SQL injection	All values are bind parameters (<code>text()</code> with <code>:params</code> , SQLAlchemy Core, <code>expanding</code> IN lists). The only dynamic SQL fragments are <code>ORDER BY</code> clauses chosen from a fixed dict by an enum-validated key, and constant <code>WHERE</code> fragments. Search input is lower-cased and <code>%</code> , <code>_</code> , <code>\</code> are escaped before binding, so wildcards match literally (tested). Import rule SQL is built from module constants; file content only ever arrives via <code>COPY</code> and parameters
CORS	The browser uses same-origin <code>/api/v1</code> via the Next.js rewrite, so CORS is not needed in normal operation. The API additionally allows only <code>CORS_ORIGINS</code> (default <code>http://localhost:3000</code>) with credentials, methods <code>GET/POST/PATCH/OPTIONS</code> and two headers
CSRF	<code>SameSite=Lax</code> stops the cookie on cross-site POST/PATCH; JSON bodies force a CORS preflight for cross-origin scripts. The multipart upload is a "simple" request, so it relies on <code>SameSite</code> alone; a same-site origin (e.g. another app on <code>localhost</code>) is not covered. Next step: <code>Origin</code> header check or double-submit token on unsafe methods
Upload hardening	ADMIN only; <code>.csv</code> extension + content-type allow-list; streamed read aborted at 25 MB (413); strict UTF-8; exact header; 64 KiB field cap; client path stripped to basename; values staged as text and never executed; SHA-256 recorded per batch
Secrets	None in Git; <code>.env</code> git-ignored; settings use <code>SecretStr</code> (not printed in logs/reprs); CI uses throw-away CI-only values; no secrets in client code (the browser never sees the API URL or any key)
Error disclosure	Generic 500 body with <code>request_id</code> ; stack traces only in server logs
Logs and PII	Access log records method, path, status, duration, <code>request_id</code> , <code>user_id</code> , never query strings, bodies, cookies or passwords. Customer ids appear in paths. Failed-login events log the attempted e-mail and client IP (needed for abuse investigation, but it is PII: production would hash or restrict it). All data is synthetic
Network	DB port bound to <code>127.0.0.1</code> ; in compose only <code>api</code> reaches <code>db</code>
Supply chain	<code>uv.lock</code> / <code>package-lock.json</code> pinned; <code>--locked</code> installs in CI; image and dependency scanning in <code>docker.yml</code> (design)

Known security gaps (also in `DECISIONS.md`): the rate limiter is per process; behind the Next.js proxy

`request.client.host` is the web container unless uvicorn is started with `--proxy-headers` and a trusted `forwarded-allow-ips`, which makes the limit effectively per e-mail; no security headers (CSP, HSTS) are set by the API itself (expected from the web tier / proxy); the code default `JWT_SECRET` is long enough to pass validation, so production must set it (a start-up guard rejecting the default when `ENVIRONMENT=production` is a one-line fix).

9. Reliability and operations

Area	Current behaviour	Production addition
Health	<code>GET /api/v1/health</code> : runs <code>SELECT 1</code> through the pool; 200 <code>healthy</code> or 503 <code>unhealthy</code> with <code>database:unavailable</code> . Used for container health and compose start ordering	Split liveness (process up) from readiness (DB + migrations at head) so a DB blip does not restart healthy pods
Logging	structlog JSON to stdout (console renderer locally); one <code>request_completed</code> line per request with <code>request_id</code> , method, path, status, <code>duration_ms</code> , <code>user_id</code> ; domain	Ship to a central store; propagate the id from the web tier; OpenTelemetry traces

	events (<code>login_failed</code> , <code>goal_updated</code> , <code>import_completed</code> with counts, <code>import_failed</code> with stack trace). Incoming <code>X-Request-ID</code> is honoured if well-formed, else generated, and echoed back	
Metrics / monitoring	Not implemented	RED metrics per route (rate, errors, p95 latency), pool saturation, import outcomes (rejected-row ratio), <code>pg_stat_statements</code> top queries, replication lag, disk; alerts on 5xx rate, p95 > SLO, failed imports, health 503
Timeouts	DB <code>statement_timeout</code> 15 s per connection; pool checkout timeout 5 s; <code>pool_pre_ping</code> discards dead connections after a DB restart	Upstream timeouts at the proxy; separate longer timeout for the import role
Connection pooling	One async engine per process: <code>pool_size</code> 10 + <code>max_overflow</code> 10	PgBouncer (transaction mode) or managed proxy when replicas x pool approaches <code>max_connections</code>
Retries	No automatic retries of writes (not idempotent in general). Reads fail fast with 500 + <code>request_id</code> ; the UI can retry GETs (TanStack Query retry for idempotent queries (<i>design</i>)). Imports are safe to retry by design: a re-upload is idempotent	Retry with backoff on serialization failures / connection resets for idempotent operations only
Import failure	Structural error: 4xx, nothing written. Merge error: the data transaction rolls back completely, batch row is <code>FAILED</code> with message, error logged with stack; staging rows are deleted in the same transaction or vanish with the rollback	Background job + progress polling for large files
Migration failure	<code>transaction_per_migration=True</code> : PostgreSQL DDL is transactional, so a failing revision rolls back entirely and <code>alembic_version</code> stays at the last good revision. The API container's endpoint stops before uvicorn starts, so a half-migrated schema is never served. Rollback is <code>alembic downgrade -1</code> , exercised in CI by a full <code>downgrade base / upgrade head</code> round trip	Expand/contract migrations for zero downtime; migration job gated before rollout; never auto-downgrade in production (restore or roll forward)
Backups	Local: named volume <code>pgdata</code> ; <code>pg_dump -Fc</code> on demand	Managed automated backups + WAL archiving for PITR (e.g. 7-35 day window), periodic restore drills into a scratch instance, logical dumps before risky migrations
Data durability nuance	<code>staging_transactions</code> is <code>UNLOGGED</code> (truncated after a crash): acceptable because it only holds in-flight batch data that is re-creatable from the uploaded file	-

10. Trade-offs, limits and roadmap

10.1 Deliberate omissions

Order execution, payments, advice, real-time prices, notifications, user management UI, SSO, multi-tenancy, Kubernetes, microservices, event bus. One modular monolith API + one web app + one database is the right size for a read-heavy internal tool with a single team.

10.2 Trade-offs taken

Decision	Gained	Given up
Natural keys as PKs	Readable data, no id mapping in imports, FK errors name real ids	Re-keying an entity is costly (would need <code>ON UPDATE CASCADE</code> or surrogate keys)
<code>VARCHAR + CHECK</code> instead of <code>PG ENUM</code>	Adding a value is a constraint swap in a transactional migration	Slightly more storage; vocabularies duplicated in Python enums (CI drift check catches model/migration mismatch, not enum/CHECK mismatch)
Plain views	Always fresh, zero orchestration	Recomputed per request: fine to $\sim 10^5$ positions (section 10.3)
Offset pagination	"Page N of M" UI, simple contract	Linear cost at depth
Cookie JWT + per-request user read	No token in JS; instant revocation	One DB lookup per request; cookie scope ties API to the web origin
Synchronous import in the request	Simple, transactional, immediate report	Request duration grows with file size (fine up to the 25 MB cap)
Snapshot-based valuation	Correct numbers from the authoritative source	Ledger and positions are not reconciled; shown as separate facts

10.3 Expected scale limits (measured, `query-plans.md`)

At the supplied volume every endpoint completes in under 5 ms of database time. On a dataset 100x-1000x larger (100k customers, 300k positions, 3M transactions):

- Ledger page for a typical customer: 1.3 ms; `count(*)` 0.2 ms (index-only). Fine.
- Customer list: **~530 ms (search) / 1.1 s (unfiltered)**, because `v_customer_aum` values every position before the page is cut. First bottleneck.
- Heavy customer (50k transactions over 2 accounts): 129 ms page 1, 293 ms at page 2,000.
- Monthly flows: 1.3 s (full-ledger aggregate).
- Single process rate limiter and single API replica.

10.4 Next steps, in order

1. **Customer list:** page customers first and value only the page (measured 530 -> 0.6 ms); a materialized `customer_aum` refreshed after each holdings load for the AUM sort.
2. **Keyset pagination** for the ledger (`((trade_date, transaction_id) < cursor)`) and per-account **LATERAL** top-N (measured 293 -> 0.6 ms at depth; 129 -> 0.45 ms page 1).
3. **Optimistic concurrency for goals:** `version` column or **If-Match:** `<updated_at>` -> 409 on a stale edit.
4. **Shared rate limiting** (Redis or gateway) and trusted proxy headers before running more than one replica; **Origin** check for unsafe methods.
5. **Background import jobs** (job table + worker, **202 Accepted** + progress polling) once files exceed what one request should hold; same stage/validate/merge SQL.
6. **Monthly-flow summary** maintained by the import merge; **range-partition** `transactions` by `trade_date` at tens of millions of rows (cheap retention, partition pruning for reports).
7. **Reconciliation:** promote "trade on inactive account" and "position vs ledger" checks into `v_data_quality_exceptions`; agree with the business whether pre-opening trades are errors.
8. **Operability:** metrics and alerts, split liveness/readiness, OpenTelemetry, least-privilege DB roles, security headers at the edge.